Projektuppgift

Ali Aldongy

MITTUNIVERSITETET
Institutionen för Informations-system och teknologi

Författare: Ali Aldongy, alal2512@student.miun.se
Utbildningsprogram: TWEUG, Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

1 Sammanfattning

Denna rapport beskriver skapandet av en interaktiv webbapplikation som har skapats med HTML, CSS och C#. Applikationen är ett frågesportspel baserat på tv-serien *The Flash.* Syftet med projektet är att använda kunskaper inom C# och skapa en fungerande webbapplikation.

Webbplatsen består av tre sidor, en startsida, en quizsida och en resultatsida. Användaren har möjlighet att välja mellan tre svårigheter, svara på frågor och samla poäng. Användaren kan dessutom välja att spara resultatet tillsammans med sitt namn när frågorna är slut. Resultaten sparas i en ny sida.

Applikationen är skapad med Razor komponenter, C# och ASP.NET Core. HTML och CSS används dessutom för sidans struktur och utformning av användargränsnittet. En separat ScoreService används för att hantera och sortera resultaten. Tjänsten kopplas till programmet med hjälp av beroendeinjektion.

2 Innehållsförteckning

1 Sammanfattning ii
2 Innehållsförteckning iii
3 Introduktion 1
3.1 Bakgrund ..... **Error! Bookmark not defined.**
3.2 Problemdefinition ..... **Error! Bookmark not defined.**
3.3 Syfte ..... **Error! Bookmark not defined.**
3.4 Mål och avgränsningar ..... **Error! Bookmark not defined.**
4 Teori 3
4.1 C# 3
4.2 .NET och ASP.NET Core 3
4.3 Razor-komponenter och Blazor 3
4.4 Objektorienterad programmering 4
4.5 Listor och LINQ 5
4.6 Händelser och databindning 5
4.7 Beroendeinjektion och tjänster 6
5 Metod 7
5.1 Arbetssätt 7
5.2 Planering av funktioner 7
5.3 Verktyg och tekniker 7
5.4 Struktur 7
6 Konstruktion 9
6.1 Applikationens struktur 9
6.2 Startsida – Home.razor 10
6.3 Quizsida – Quiz.razor 10
6.3.1 Val av svårighetsgrad 10
6.3.2 Samtliga frågor 11
6.3.3 Poängräkning 11
6.3.4 Avslutning av quizet 12
6.3.5 Omstart 12
6.4 Resultatsida – Score.razor 13
6.5 Resultattjänsten – ScoreService.cs 14
6.6 Programkonfiguration – Program.cs 14
6.7 Användargränssnitt och grafisk design 15
6.8 Flödesschema 15
7 Resultat 17
8 Slutsatser 19
9 Källförteckning 21

3 Introduktion

3.1 Bakgrund

Webbapplikation är något som idag används väldigt ofta. Allt från utbildning och underhållning till att sprida information. Interaktiva webbapplikationer ger personer möjlighet att få en direkt återkoppling på sina handlingar. Frågesport är ett exempel på en sådan applikation då information och resultat kombineras.

Detta projekt har använts som en webbplats där användare kan testa sina kunskaper om tv serien *The Flash.* Frågorna rör allt från karaktärer och händelser till krafter och andra delar av berättelsevärlden. Webbplatsen är skapad för att vara underhållande och enkel att använda.

3.2 Problemdefinition

Meningen med projektet är att skapa en webbplats där användare (främst *The Flash* fans) ska ha möjlighet ta del av en frågesport och se sitt resultat. För att uppnå detta mål måste flera funktioner uppnås. De största problemen som projektet ska lösa är:

- Användare ska kunna starta quiz på ett tydligt sätt.
- Frågorna ska delas in utifrån svårighetsgrad.
- Applikationen ska räkna antal poäng automatiskt.
- Användaren ska vid varje fråga kunna se hur många frågor som besvarats samt hur många poäng som uppnåtts.
- Resultaten ska kunna visas i en separat sida.

Dessa funktioner kommer bli svåra att hantera utan en tydlig struktur. Därför har flera Razor komponent valts, istället för att all logik placeras i en stor kod del.

3.3 Syfte

Syftet med detta projekt är att skapa en enkel interaktiv webbapplikation med C# och .NET och använda kunskaper som har lärts under kursens gång. Ett annat syfte är att få bättre koll på hur en applikation kan delas upp i olika delar med Razor komponenter.

3.4 Mål och avgränsningar

Målet med projektet är att skapa en webbplats med dessa funktioner:

- En fängslande startsida med introduktion till quizet.
- En quizsida med tre svårighetsgrader: Easy, Intermediate och Advanced.
- Flervalsfrågor med fyra alternativ för samtliga frågor.
- Automatisk poängberäkning.
- Möjlighet att spara resultatet i en separat sida.

Projektuppgift
Ali Aldongy 2026-10-24

Applikationen innehåller totalt 30 frågor, 10 frågor för varje svårighetsgrad. Projektet är avgränsat till en enklare webbapplikation. Det finns ingen användarregistrering, ingen inloggning och ingen databas. Resultaten lagras tillfälligt i programmets minne genom en lista i ScoreService.

4 Teori

4.1 C#

C# är ett programmeringsspråk som används tillsammans med .NET. Med C# kan man skapa olika program, bland annat webbplatser, datorprogram och tjänster. Språket har funktioner som klasser, metoder och samlingar som hjälper till att organisera koden [1]. I detta quiz används C# för att styra hur programmet fungerar. Koden bestämmer vilka frågor som ska visas, kontrollerar om spelarens svar är rätt och räknar poängen.

Ett viktigt begrepp i C# är variabler. En variabel används för att spara ett värde som programmet kan använda eller ändra. I quizzet används variabler för att hålla reda på spelarens poäng och vilken fråga som visas.

```
private int currentQuestionIndex = 0;
6 references
private int score = 0;
```

Variabeln CurrentQuestionIndex anger indexet för den aktuella frågan. Variabeln Score anger antalet korrekta svar. Båda variablerna startar från noll.

C# använder dessutom metoder för att samla kod som utför en viss uppgift. I projektet används metoder för att välja svårighetsgrad, kontrollera svar, starta om quizet och spara resultat.

4.2 .NET och ASP.NET Core

.NET är en plattform som innehåller verktyg och bibliotek för att skapa och köra program[2]. C# är ett av programmeringsspråken som används tillsammans med .NET.

ASP.NET Core är ett ramverk inom .NET som används för att skapa webbapplikationer och webbtjänster [3]. Det hjälper till att hantera hur webbapplikationen fungerar, hur den tar emot förfrågningar samt vilka delar av programmet som ska användas.

I projektet används ASP.NET Core för att starta webbapplikationen och ställa in hur den ska fungera. Detta görs främst i filen Program.cs. I den filen registreras tjänster, inställningar för hur förfrågningar hanteras görs och Razor-komponenterna kopplas till webbapplikationen.

4.3 Razor komponenter och Blazor

Razor är en teknik som gör det möjligt att blanda HTML och C#-kod [4]. Blazor använder Razor-komponenter för att skapa interaktiva användargränssnitt[5]. En Razor-komponent kan innehålla både kod och det som visas på skärmen. Den kan också ha en egen webbadress, hantera användarens klick och uppdatera innehållet när något förändras i programmet.

I *The Flash* quizzet används tre huvudsakliga komponenter:

- Home.razor som visar startsidan.
- Quiz.razor som används när användaren tar själva quizzet.
- Scores.razor som visar användaren resultat om dem sparar det.

@page bestämmer vilken webbadress som leder till en viss komponent. I projektet används @page "/" för startsidan och @page "/quiz" för quizsidan. Blazor har olika sätt att visa och hantera innehåll. I projektets fil Program.cs används AddInteractiveServerComponents() och AddInteractiveServerRenderMode() för att aktivera interaktiv serverrendering. Det innebär att C#-koden körs på servern och att användaren kan interagera med sidan genom att bland annat klicka på knappar och få innehållet uppdaterat.

4.4 Objektorienterad programmering

Objektorienterad programmering innebär att man organiserar koden med hjälp av klasser och objekt [6]. En klass beskriver vilka egenskaper och funktioner ett objekt har. I quizzet används klassen QuestionModel för att beskriva en quizfråga och lagra information om den.

```
public class QuestionModel
{
    31 references
    public string QuestionText { get; set; } = "";
    31 references
    public List<string> Options { get; set; } = new List<string>();
    31 references
    public string CorrectAnswer { get; set; } = "";
    31 references
    public string Difficulty { get; set; } = "";
}
```

Klassen innehåller fyra egenskaper:

- QuestionText lagrar frågans text.
- Options innehåller de olika svarsalternativen.
- CorrectAnswer sparar det rätta svaret.
- Difficulty anger vilken svårighetsgrad frågan tillhör.

Genom att samla dessa egenskaper i en klass kan varje fråga skapas som ett eget objekt. Flera frågor kan sedan samlas i en lista.

I projektet används också klassen ScoreModel för att spara information om spelarens resultat. Den innehåller spelarens namn, svårighetsgrad, poäng och det totala antalet frågor. Genom att använda klasser blir informationen lättare att organisera och hantera i programmet.

4.5 Listor och LINQ

En lista används för att samla flera objekt på ett ställe. I C# används till exempel List<QuestionModel> för att lagra frågor och List<ScoreModel> för att lagra resultat.

I quizzet används en lista för att samla alla frågor. När spelaren väljer svårighetsgrad skapas en ny lista som bara innehåller frågor som är vald till den nivån.

```
filteredQuestions = questions
.Where(QuestionModel q => q.Difficulty == difficulty)
.ToList();
```

Här används LINQ, som är en funktion i .NET som gör det enklare att söka efter, filtrera och hantera information i listor [7].

Metoden Where väljer ut de frågor som har samma svårighetsgrad som spelaren har valt.

Metoden ToList samlar sedan de valda frågorna i en lista.

På resultatsidan används LINQ för att sortera resultaten:

```
return _highscores
    .OrderByDescending(s => s.Score)
    .ThenBy(s => s.PlayerName)
    .ToList();
```

Först sorteras resultaten efter poäng i fallande ordning. Om flera resultat har samma poäng sorteras de efter spelarens namn i stigande ordning.

4.6 Händelser och Databindning

En interaktiv applikation ska kunna reagera på vad användaren gör. I Blazor kan knappar kopplas till C#-metoder med hjälp av @onclick.

I quizet används detta för att låta spelaren välja svårighetsgrad och svarsalternativ. När spelaren klickar på en knapp körs en metod som utför den uppgift som knappen är kopplad till.

```
<button @onclick="() => CheckAnswer(option)">@option</button>
```

När användaren klickar på knappen körs metoden CheckAnswer som får det valda svaret som argument.

Databindning används även i projektet. Med @bind kopplas fältet där spelaren skriver sitt namn till en variabel. Då sparas namnet automatiskt i variabeln när användaren skriver in det.

```
<input type="text" @bind="playerName" placeholder="Enter your name..."  class="name-input">
```

När användaren skriver sitt namn i fältet uppdateras variabeln playerName automatiskt. Namnet kan sedan användas när spelarens resultat ska sparas.

4.7 Beroendeinjektion och tjänster

Beroendeinjektion (dependency injection) innebär att en del av programmet får de tjänster den behöver utan att själv behöva skapa dem [8]. I quizzet används tjänsten ScoreService för att hantera spelarens resultat. Tjänsten registreras i filen Program.cs.

```
builder.Services.AddSingleton<ScoreService>();
```

De sidor som behöver behöver tjänsten kan använda den genom injektion

```
@inject ScoreService ScoreService
```

Detta innebär att flera olika tjänster kan använda samma tjänst. I projektet använder quiz sidan den för att spara resultatet, medan resultatsidan använder den för att visa resultaten.

```
  private readonly List<ScoreModel> _highscores = new();
```

I quizzet lagras resultaten i en vanlig C# lista i ScoreService. Resultaten sparas i programmets minne.

5 Metod

5.1 Arbetssätt

Projektet har gjorts som en uppgift där målet har varit att skapa en fungerande applikation och använda det vi lärt oss under kursen. Arbetet har delats upp i flera delar:

- Planering av webbplatsens funktioner och tema.
- Skapande av startsidan och navigeringen.
- Utveckling av quizfrågor och olika svårighetsgrader.
- Implementering av poängräkning och möjlighet att starta om quizet.
- Skapande av en tjänst för att spara resultat och visa en topplista.
- Utformning av CSS och webbplatsens utseende.
- Testning av funktionerna för att kontrollera att allt fungerar.

Genom att dela upp arbetet blev det enklare att fokusera på en del i taget. Efteråt sattes alla delar ihop till en fungerande applikation.

5.2 Planering av funktioner

Tanken är att användaren först öppnar startsidan och sedan går vidare till quizet. Där väljer användaren svårighetsgrad, svarar på frågorna och sparar sitt resultat när quizet är klart. På resultatsidan kan användaren se de resultat som har sparats medan programmet är igång. Applikationen är uppdelad på tre olika sidor i stället för att ha allt på en och samma sida. Det gör det enklare att navigera på webbplatsen och förstå hur den är uppbyggd.

5.3 Verktyg och tekniker

I projektet används flera olika tekniker för att bygga quizzet. C# används för quizets logik, poängberäkning och hantering av resultat. .NET och ASP.NET Core används för att köra webbapplikationen, medan Blazor och Razor-komponenter används för att bygga sidorna, navigeringen och funktionerna som användaren kan interagera med. HTML används för att strukturera innehållet och CSS för att styra webbplatsens utseende. LINQ används för att filtrera och sortera listor, och beroendeinjektion gör det möjligt för olika komponenter att använda ScoreService för att hantera resultaten. Visual Studio Code (VS Code) används som utvecklingsmiljö för att skriva, organisera och arbeta med projektets kod.

5.4 Struktur

För att göra koden enklare att arbeta med har informationen delats upp i två delar, frågor och resultat. Varje fråga innehåller själva frågan, svarsalternativen, rätt svar och svårighetsgrad. Ett resultat innehåller spelarens namn, vald

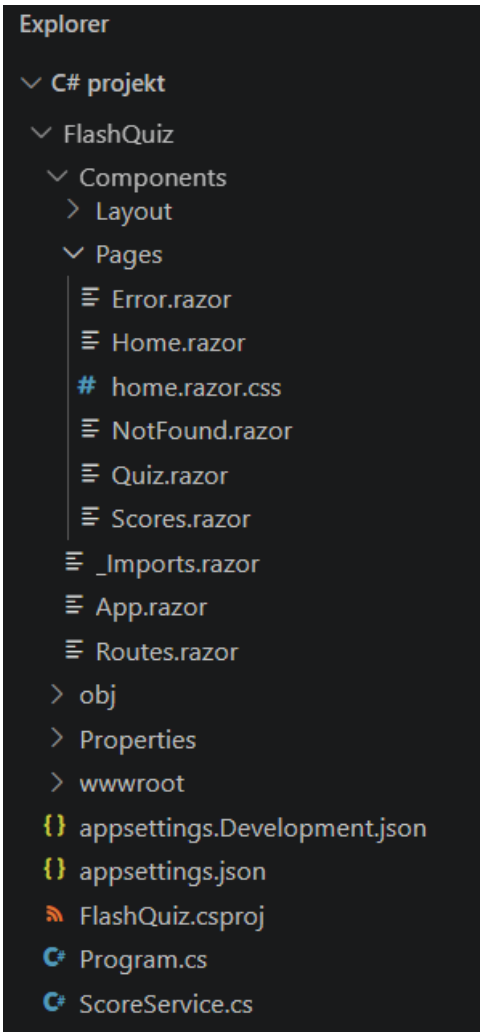
svårighetsgrad, poäng och hur många frågor quizet innehöll. Detta gör att alla frågor kan hanteras på samma sätt. Resultaten kan också sorteras och visas på resultatsidan utan att sidan behöver veta hur spelaren genomförde quizet.

6 Konstruktion

6.1 Applikationens struktur

Quizzet består av tre Razor komponenter och en separat tjänst som hanterar resultaten. De tre komponenterna ansvarar för startsidan, quizzet och resultatsidan. Tjänsten används för att spara och hämta resultat. I Program.cs ställs applikationen in och tjänsten registreras så att komponenterna kan använda den.

Strukturen kan sammanfattas så här:



Strukturen visar att varje del har sin egen uppgift. Quizlogiken finns mestadels i Quiz.razor, medan hanteringen av resultat ligger i en separat tjänst. Det gör koden enklare att förstå och hålla ordning på.

6.2 Startsida – Home.razor

Startsidan är det första användaren ser när webbplatsen öppnas. Den innehåller en rubrik, en kort introduktion och en knapp som tar användaren vidare till quizet. Rubriken ”Feel the Speed. Become The Flash.” passar webbplatsens tema.
Texten förklarar kort vad quizet går ut på och uppmuntrar användaren att börja spela. I bakgrunden finns två rörande bilder som visar The Flash och Zoom. Med hjälp av CSS placeras bilderna bredvid varandra och mörkas ned. En gradientmask används för att skapa en mjuk övergång mellan bilderna. Klassen .main-header använder flexbox för att placera innehållet i mitten av sidan. Klassen .intro-content samlar rubriken, texten och startknappen i en vertikal layout, så att allt hamnar under varandra.

Navigeringen till quizsidan görs med en NavLink

```
<NavLink href="quiz" class="start-btn">
    Go Full Speed! 
</NavLink>
```

Länken ser ut som en knapp men används för att navigera till quizsidan. När användaren klickar på den kommer hen vidare till quizet. Startsidan hanterar ingen poängberäkning. Den ska i stället presentera webbplatsen och göra det enkelt för användaren att komma igång med quizet.

6.3 Quizsida – Quiz.razor

Quizzsidan är den viktigaste delen av applikationen. Här väljer användaren svårighetsgrad, svarar på frågorna och får sina poäng räknade utifrån svaren.

6.3.1 Val av svårighetsgrad

När quizzsidan öppnas är selectedDifficulty tom. Programmet kontrollerar detta med:

```
@if (string.IsNullOrEmpty(selectedDifficulty))
{
    <div class="difficulty-selection">
        <h2 class="difficulty-title">Choose Your Speed</h2>
        <button @onclick='() => SelectDifficulty("easy")' class="diff-btn">Easy</button>
        <button @onclick='() => SelectDifficulty("intermediate")' class="diff-btn">Intermediate</button>
        <button @onclick='() => SelectDifficulty("advanced")' class="diff-btn">Advanced</button>
    </div>
}
```

Så länge användaren inte har valt någon svårighetsgrad visas tre knappar: Easy, Intermediate och Advanced. När användaren klickar på en av knapparna körs metoden SelectDifficulty:

```
private void SelectDifficulty(string difficulty)
{
    selectedDifficulty = difficulty;
    filteredQuestions = questions
    .Where(QuestionModel q => q.Difficulty == difficulty)
    .ToList();
    currentQuestionIndex = 0;
    score = 0;
}
```

Metoden sparar den valda svårighetsgraden och väljer ut de frågor som hör till den nivån. Den återställer också frågeindexet och poängen så att quizet börjar från början. I koden finns tio frågor för varje svårighetsgrad. Easy innehåller enklare frågor, Intermediate har frågor med lite mer detaljer och Advanced innehåller svårare frågor om exempelvis olika speedsters, skurkar och händelser i berättelsen.

6.3.2 Samtliga frågor

När användaren har valt svårighetsgrad visas en fråga i taget tillsammans med de olika svarsalternativen. Frågan hämtas med hjälp av currentQuestionIndex, som håller reda på vilken fråga som visas. Användaren kan också se hur många frågor quizet innehåller och hur många poäng hen har fått hittills.

```
<p>Question @(currentQuestionIndex + 1)/@filteredQuestions.Count <span>Score: @score</span></p>

<h2>@filteredQuestions[currentQuestionIndex].QuestionText</h2>
```

I koden börjar indexet på noll, men för användaren visas frågorna från nummer ett. Det gör det enklare att förstå vilken fråga man är på.

```
@foreach (var string option in filteredQuestions[currentQuestionIndex].Options)
{
    <button @onclick="() => CheckAnswer(option)">@option</button>
}
```

Svarsalternativen hämtas från listan Options för den aktuella frågan.
Programmet går igenom listan och skapar en knapp för varje svarsalternativ.

6.3.3 Poängräkning

När användaren väljer ett svar anropas metod CheckAnswer:

```
private void CheckAnswer(string selectedOption)
{
    if (selectedOption == filteredQuestions[currentQuestionIndex].CorrectAnswer)
    {
        score++;
    }

    currentQuestionIndex++;
}
```

Metoden jämför användarens svar med frågans CorrectAnswer. Om svaret är rätt får användaren en poäng. Oavsett om svaret är rätt eller fel går quizet vidare till nästa fråga. Eftersom quizet innehåller tio frågor kan användaren få högst tio poäng. Fel svar ger inga poäng, och det finns varken tidsbonus eller minuspoäng. Programmet använder sedan frågeindexet för att kontrollera om det finns fler frågor kvar eller om quizet är färdigt.

6.3.4 Avslutning av quizzet

När currentQuestionIndex når slutet av frågelistan visas avslutningssidan. Där ser användaren hur många poäng hen fick. Det finns också ett fält där spelaren kan skriva sitt namn och spara resultatet. Om namnfältet inte är tomt eller bara innehåller mellanslag kan resultatet sparas:

```
private void SaveAndNavigate()
{
    if (!string.IsNullOrWhiteSpace(playerName))
    {
        ScoreService.AddScore(playerName, selectedDifficulty, score, filteredQuestions.Count);
        Navigation.NavigateTo(uri: "/Scores");
    }
}
```

Metoden använder IsNullOrWhiteSpace för att kolla att användaren har skrivit ett namn och att fältet inte bara innehåller mellanslag. När resultatet har sparats skickas användaren vidare till resultatsidan. Om användaren försöker spara utan att skriva ett giltigt namn visas inget felmeddelande, utan inget händer.

6.3.5 Omstart

Metoden RestartQuiz återställer quizet till början:

```
private void RestartQuiz()
{
    currentQuestionIndex = 0;
    score = 0;
    selectedDifficulty = "";
}
```

Index och poäng återställs, och den valda svårighetsgraden töms. Därefter visas valet av svårighetsgrad igen. På så sätt kan användaren spela quizet på nytt utan att behöva ladda om hela webbplatsen.

6.4 Resultatsida – Score.razor

Resultatsidan visar sparade resultat i en tabell. Tabellen innehåller spelarens placering, namn, svårighetsgrad och poäng. När sidan öppnas hämtas resultaten från ScoreService:

```
protected override void OnInitialized()
{
    highscores = ScoreService.GetHighscores();
}
```

Resultaten sparas i en lista som används för att visa tabellen. Om listan är tom visas ett meddelande om att det inte finns några resultat än. Om det finns resultat går programmet genom listan och skapar en rad i tabellen för varje spelare.
Den som ligger på första plats får CSS-klassen top-player, som gör att spelarens rad får ett annat utseende. Ordningen på resultaten bestäms av tjänsten och inte av tabellen. Det gör att koden för sortering hålls separat från den del som visar resultaten.

```
private string GetOrdinalSuffix(int num)
{
    if (num <= 0) return "";

    if ((num % 100) >= 11 && (num % 100) <= 13)
    {
        return "th";
    }

    return (num % 10) switch
    {
        1 => "st",
        2 => "nd",
        3 => "rd",
        _ => "th"
    };
}
```

Metoden GetOrdinalSuffix lägger till rätt ändelse till placeringarna, t.ex 1st, 2nd, 3rd och 4th. Den tar också hänsyn till talen 11–13, som alla får ändelsen th. Resultatsidan har samma stil som startsidan och quizsidan, med mörk bakgrund, ljus text och gula detaljer. Det gör att hela webbplatsen får ett enhetligt utseende.

6.5 Resultattjänsten - ScoreService.cs

ScoreService är en separat C#-klass som används för att spara och hämta resultat. Klassen innehåller ScoreModel, som beskriver hur ett resultat ser ut, och en privat lista där alla resultat sparas. När användaren har avslutat quizzet anropar quizzsidan metoden AddScore. Den skapar ett nytt resultat och lägger till det i listan.

```
public void AddScore(string name, string difficulty, int score, int total)
{
    _highscores.Add(new ScoreModel
    {
        PlayerName = name,
        Difficulty = difficulty,
        Score = score,
        TotalQuestions = total
    });
}
```

Metoden tar emot fyra värden och sparar dem i rätt egenskaper i resultatobjektet. När resultatsidan behöver visa resultaten anropas GetHighscores. Metoden returnerar en ny lista där resultaten sorteras efter poäng och sedan efter namn.

6.6 Programkonfiguration – Program.cs

Filen Program.cs innehåller inställningarna för webbapplikationen. Här skapas applikationen, tjänster registreras och det bestäms hur förfrågningar ska hanteras.
En viktig del av koden är:

```
builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();

builder.Services.AddSingleton<ScoreService>();
```

Först registreras de tjänster som behövs för Razor-komponenterna och för att göra dem interaktiva på servern. Därefter registreras ScoreService som en singleton, vilket innebär att samma instans används av de komponenter som behöver tjänsten.

Resten av filen innehåller inställningar för bland annat HTTPS, hantering av fel och hur Razor-komponenterna kopplas till webbapplikationen.

```
app.MapRazorComponents<App>()
    .AddInteractiveServerRenderMode();

app.Run();
```

Här kopplas komponenterna till webbapplikationen och interaktiviteten på servern aktiveras. Med app.Run() startas applikationen. Detta gör att webbplatsens komponenter kan köras med ASP.NET Core och använda de tjänster som har registrerats, bland annat ScoreService.

6.7 Användargränssnitt och grafisk design

Utseendet är en viktig del av quizzet eftersom alla sidor ska passa ihop och följa

samma tema. Färgerna som används mest är svart, mörkgrått, rött och gult. Gult används bland annat på knappar och markeringar, medan rött syns i skuggor och när man hovrar över olika element. Det gör att sidorna får ett enhetligt utseende. På startsidan används animerade GIF-bilder för att göra den mer levande.

Quizzsidan har en mörk bakgrund som gör att frågorna och svarsknapparna syns tydligare.

På resultatsidan visas resultaten i en tabell så att de blir enkla att jämföra.

I CSS används bland annat flexbox för att placera innehållet, object-fit för att anpassa bilder och övergångar och transformeringar för att skapa effekter på knapparna. Media queries används dessutom för att anpassa layouten efter olika skärmstorlekar. Det gör att sidan fungerar bättre på både datorer och mobiltelefoner.

6.8 Flödesschema

Flödesschemat nedan visar hur användaren går genom quizzet, från startsidan och valet av svårighetsgrad till att besvara frågorna, spara sitt resultat och se topplistan.

```
N1["1. Visa startsidan"]
N1 --> N2["2. Användaren startar quizet"]
N2 --> N3["3. Välj svårighetsgrad"]
N3 --> N4["4. Hämta frågor för vald nivå"]
N4 --> N5["5. Visa fråga och svarsalternativ"]
N5 --> N6["6. Användaren väljer ett svar"]
N6 --> N7{"7. Är svaret rätt?"}

N7 -- Ja --> N8["8. Öka poängen med 1"]
N7 -- Nej --> N9["8. Behåll poängen"]

N8 --> N10["9. Gå vidare till nästa fråga"]
N9 --> N10
N10 --> N11{"10. Finns fler frågor?"}

N11 -- Ja --> N5
N11 -- Nej --> N12["11. Visa slutpoängen"]

N12 --> N13["12. Användaren anger sitt namn"]
N13 --> N14{"13. Saknas någon input?"}

N14 -- Nej --> N15[" Spara resultat i ScoreService "]
```

```
N15 --> N13
   N14 -- Ja --> N16["14. Inget händer"]

   N16 --> N17["15. Visa resultatsidan"]
   N17 --> E([Slut])
```

7 Resultat

Resultatet av arbetet är en webbapplikation med tre sidor och funktioner som tillsammans gör att användaren kan spela ett quiz.

Startsidan presenterar webbplatsen och leder vidare till quizet.

På quizsidan kan användaren välja svårighetsgrad, svara på frågor, samla poäng och spara sitt resultat.

Resultatsidan visar de sparade resultaten i en sorterad tabell. Det finns totalt 30 frågor i koden, med tio frågor för varje svårighetsgrad. Varje fråga har fyra svarsalternativ och ett rätt svar.

I projektet används C# för logiken, Razor-komponenter för sidorna och CSS för utseendet.

En separat tjänst, ScoreService, används för att hantera resultaten.

Projektet visar hur olika delar av .NET kan användas tillsammans för att skapa en fungerande webbapplikation.

För att kontrollera att applikationen fungerar som den ska behöver man testa funktionerna på den färdiga webbplatsen. Följande testfall kan användas för att se att allt fungerar som det ska.

Test	**Genomförande**	**Förväntat resultat**
Startsida	Öppna webbplatsen	Startsidan visas
Navigering	Klicka på startlänken	Quizzsidan öppnas
Svårighetsgrad	Välj Easy	Bara Easy-frågorna visas
Svårighetsgrad	Välj Intermediate	Bara Intermediate-frågorna visas
Svårighetsgrad	Välj Advanced	Bara Advanced-frogrona visas
Rätt svar	Välj ett korrekt svar	Poängen ökar +1
Fel svar	Välj ett felaktigt svar	Poängen stannar samma
Frågeflöde	Svara på alla frågor	Nästa fråga visas efter varje svar
Quizzavslut	Svara på sista frågan	Slutresultat visas
Tomt namn	Försök spara utan namn	Inget händer
Giltigt namn	Ange namn och spara	Resultatet läggs till och finns med i resultatsidan
Sortering	Gör quizzet flera gånger med olika poäng	Högre poäng ska vara högre upp
Omstart	Klicka på Play again	Poäng och frågor

		återställs
Tom topplista	Öppna resultatsida utan att ha gjort quiz	Tom lista visas
Omstart av server	Registrera resultat och starta om programmet	Resultaten ska försvinna

Utifrån koden går det att se hur de viktigaste funktionerna fungerar. Frågorna filtreras med LINQ, svaren jämförs med rätt svar och poängen uppdateras beroende på om svaret är rätt eller fel. När alla frågor är svarade visas slutresultatet. När användaren sparar sitt resultat kontrolleras att namnet inte är tomt. Om det finns ett namn skickas informationen till ScoreService, som skapar ett nytt resultat och sparar det i listan.

De viktigaste funktionerna i projektet finns med i koden. Webbplatsen har en startsida, en quizsida och en resultatsida. Användaren kan välja mellan tre svårighetsgrader, svara på frågor, samla poäng och spara sitt resultat. I koden används också objekt, listor, metoder, villkorssatser och LINQ. Genom att ha en separat tjänst för resultaten blir det enklare att hålla isär logiken och sidornas utseende.

8 Slutsatser

Projektet visar hur man kan använda C# och .NET för att skapa en webbapplikation med flera funktioner. Med Razor-komponenter, C# och CSS har jag byggt en webbplats där användaren kan svara på frågor och spara sitt resultat.
Webbplatsen är uppdelad i tre sidor: en startsida, en quizsida och en resultatsida. Det gör det enklare att hålla ordning på funktionerna och gör webbplatsen lättare att använda. Jag har också skapat en separat klass, ScoreService, som hanterar resultaten.
Projektet visar hur C# används för att styra quizet, hur frågor lagras i listor och objekt samt hur LINQ används för att filtrera och sortera information. Med beroendeinjektion kan olika delar av applikationen använda samma tjänst för att hantera resultaten.

Det finns samtidigt några saker som kan förbättras. Resultaten sparas bara i minnet och försvinner när applikationen startas om. Det skulle också vara bra att lägga till tydligare felmeddelanden och fler funktioner för att hantera resultaten.

Sammanfattningsvis har projektet gett mig praktisk erfarenhet av att bygga en webbapplikation och använda flera av de tekniker som ingår i kursen. Det finns en fungerande grund att bygga vidare på, exempelvis genom att lägga till en databas och förbättra användarupplevelsen.

När man utvecklar en interaktiv applikation är det viktigt att alla delar fungerar tillsammans. I mitt quiz behöver frågeindexet, poängen, svårighetsgraden och hanteringen av resultat fungera som de ska. Det är också viktigt att kontrollera att programmet inte försöker visa en fråga som inte finns.
I den nuvarande koden kontrolleras frågeindexet innan frågan visas. När alla frågor är besvarade visas i stället avslutningsvyn.

En annan viktig del är hur resultaten sparas. Med en singleton-tjänst används samma lista av de komponenter som delar tjänsten. Det gör lösningen enkel, men resultaten sparas bara i minnet och försvinner när applikationen startas om.

Om applikationen utvecklas vidare och fler användare ska kunna använda den skulle en databas vara ett bättre alternativ för att spara och hantera resultaten.

En förbättring hade varit att dela upp koden mer. Just nu finns frågemodellen och alla frågor direkt i Quiz.razor. Det fungerar bra i ett mindre projekt, men om fler frågor och funktioner läggs till kan koden bli svårare att hålla ordning på. Man skulle kunna skapa en separat tjänst för frågorna som hämtar och filtrerar dem. Då skulle Quiz.razor främst hantera det som användaren gör på sidan.
Det hade också varit bra att lägga till tydligare felmeddelanden. Om användaren försöker spara sitt resultat utan att ange ett namn skulle ett meddelande visas

som förklarar vad som behöver fyllas i. En annan förbättring hade varit att använda en databas för resultaten. Då skulle de finnas kvar även efter att applikationen startats om.

9 Källförteckning

[1] W3schools, ”C# Tutorial”,
https://www.w3schools.com/cs/index.php
Hämtad 2026-10-20

[2] Microsoft, ” The modern, innovative, open-source programming language for building all your apps.”,
https://dotnet.microsoft.com/en-us/languages/csharp
Hämtad 2026-10-20

[3] Microsoft, ”Overview of ASP.NET Core”,
https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-10.0
Uppdaterad 2026-07-22 Hämtad 2026-10-20

[4] Microsoft, ”Razor Syntax reference for ASP.NET Core”,
https://learn.microsoft.com/en-us/aspnet/core/mvc/views/razor?view=aspnetcore-10.0
Uppdaterad 2026-08-26 Hämtad 2026-10-20

[5] Microsoft, ”What is Blazor?”,
https://dotnet.microsoft.com/en-us/apps/aspnet/web-apps/blazor
Hämtad 2026-10-20

[6] Dbwebb, ”Objektorienterad programmering”,
https://dbwebb.se/guide/kom-igang-med-objektorienterad-programmering-i-php/objektorienterad-programmering
Hämtad 2026-10-20

[7] Microsoft, ”Introduction to LINQ Queries in C#”,
https://learn.microsoft.com/en-us/dotnet/csharp/linq/get-started/introduction-to-linq-queries
Uppdaterad 2025-03-24 Hämtad 2026-10-20

[8] Microsoft, ”.NET dependency injection”,
https://learn.microsoft.com/en-us/dotnet/core/extensions/dependency-injection/overview
Uppdaterad 2026-01-28 Hämtad 2026-10-20